

Planilla de cálculo

=SUMA(A2; A3)

Parámetros

↓
Devuelve resultado
de la suma

Programación I

Funciones

Funciones

Una función es un conjunto de instrucciones que realizan una tarea específica de manera independiente del resto del programa. Se caracterizan por tener un nombre, un valor devuelto y parámetros.

- **Nombre:** El nombre de una función debe obedecer las reglas para nombrar variables.
- **Valor devuelto:** En su tarea a resolver, una función puede devolver un valor (entero, float, char, etc) o no devolver nada (void).
- **Parámetros:** Para poder resolver su tarea, una función puede recibir ninguno o varios parámetros. Estos se guardarán en variables que serán locales a la función.

Funciones

■ **Llamado**: Cuando se ejecuta la función y realiza la tarea para la que fue desarrollada.

■ **Declaración**: Se determinan el valor devuelto de la función, el nombre de la misma y los parámetros que recibe. Suelen aparecer en archivos H.

■ **Definición**: Se determinan el valor devuelto de la función, el nombre de la misma, los parámetros que recibe y el código que representa el algoritmo de la función. Suelen aparecer en archivos CPP.

Llamado a una función

Ejemplo de uso de una función llamada *sumar* desde la función *main*

```
#include <iostream>
using namespace std;
int main(){
    int n1, n2, suma;
    cin >> n1;
    cin >> n2;
    suma = sumar(n1, n2);
    cout << endl << suma;
    return 0;
}
```



Llamado a la función *sumar*

- El objetivo de la función **sumar** es sumar dos números.
- Recibe como dos números enteros que sumará
- Devuelve como valor de retorno un valor **int** con el resultado de la suma.

Declaración

```
tipo nombreFuncion (parámetros);
```

La **declaración** de una función debe determinar el tipo de dato que devuelve primero.

Luego cómo se llamará la función.

Por último, los tipos de datos de los parámetros que recibirá separados por coma. Si no recibe parámetros no se pone nada entre los paréntesis.

Declaración de una función

Un ejemplo de una declaración de función puede ser la siguiente:

```
bool es_primo(int);
```

Declaración de función que recibe un entero y devuelve un valor bool.

```
bool suma(int nro1, int nro2);
```

Declaración de función que recibe dos enteros y devuelve un valor entero. También es válido nombrar la/s variable/s de los parámetros.

```
void mostrar_primos_entre(int, int);
```

Declaración de función que recibe dos enteros y no devuelve nada.

Definición

```
tipo nombreFuncion (parámetros) {  
    // Código de la función  
}
```

La **definición** de una función es similar a la **declaración** pero se nombran las variables en los parámetros y se establece lo que hace la función entre las llaves de la misma.

Definición de una función

```
int sumar(int nro1, int nro2){  
    int resultado;  
    resultado = nro1 + nro2;  
    return resultado;  
}
```

Definición de la función *sumar*

- La función recibe los parámetros enteros y los declara como **nro1** y **nro2**.
- La función declara la variable local **resultado** para realizar su tarea.
- La función **devuelve** el contenido de resultado que contiene la suma de ambos números.

Ámbito de variables

Una variable es local al **ámbito** en el que fue declarada. El ámbito está dado por las llaves. Fuera de ellas la variable no existe.

Ámbito de variables

```
#include <iostream>
using namespace std;
int main(){
    int n1, n2, suma;
    cin >> n1;
    cin >> n2;
    suma = sumar(n1, n2);

    cout << endl << suma;
    return 0;
}
```

```
int sumar(int nro1, int nro2){
    int resultado;
    resultado = nro1 + nro2;
    return resultado;
}
```

n1 <-- 100

n2 <-- 400

suma <-- 500

~~nro1 <-- 100~~

~~nro2 <-- 400~~

~~resultado <-- 500~~

Retornar valores

Una función puede retornar un valor o ninguno. Esto se logra mediante la palabra reservada **return**.

```
#include <iostream>
using namespace std;
int main(){
    int n1, n2, suma;
    cin >> n1;
    cin >> n2;
    suma = sumar(n1, n2);

    cout << endl << suma;
    return 0;
}
```

```
int sumar(int nro1, int nro2){
    int resultado;
    resultado = nro1 + nro2;
    return resultado;
}
```

Parámetros

Una función puede recibir ninguno, uno o varios parámetros. Los parámetros recibidos deben permitir a la función procesarlos y asistir en la tarea que la función debe realizar.

Un parámetro a una función puede ser recibido por **valor**, **referencia** o **dirección**.

Parámetros por valor

Un parámetro enviado por valor permite que la función reciba una copia del valor de la variable/constante original o bien un valor constante explícito.

Al ser una copia del valor, cualquier cambio realizado al contenido de la variable recibida no afectará al valor de la variable original.

```
int main(){  
    int bart = 1000;  
    mi_funcion(bart);  
    cout << bart; // Mostrará 1000  
}
```

```
void mi_funcion(int bort){  
    cout << bort; // Mostrará 1000  
    bort = 10;  
    cout << bort; // Mostrará 10  
}
```

Parámetros por referencia

Un parámetro enviado por referencia permite que la función reciba la variable original. Es decir, un alias o sinónimo de la variable original.

Al ser una referencia, cualquier cambio realizado al contenido de la variable recibida afectará al valor de la variable original.

```
int main(){
    int bart = 1000;
    mi_funcion(bart);
    cout << bart; // Mostrará 10
}
```

```
void mi_funcion(int &bort){
    cout << bort; // Mostrará 1000
    bort = 10;
    cout << bort; // Mostrará 10
}
```

Funciones

Ventajas de utilizar funciones:

- Dividir un problema grande en pequeños problemas más chicos. Abstrayendo esa tarea del problema general a resolver.
- Dar legibilidad al código, haciendo algoritmos más fáciles de entender y mantener.
- Dar capacidad de reutilización al código. Haciendo funciones que resuelven tareas únicas aumenta la posibilidad de poder utilizarlas nuevamente.
- Evita la repetición de código. Cada vez que necesito realizar la misma tarea puedo llamar nuevamente a la función.
- Modificar el funcionamiento de un programa o encontrar errores es más fácil si tenemos la lógica separada en funciones.